1

Introducción

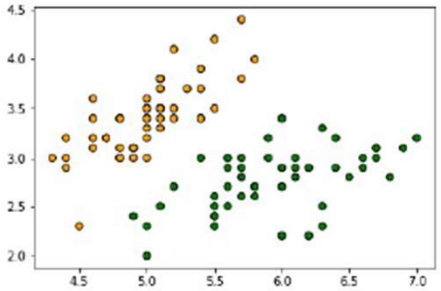
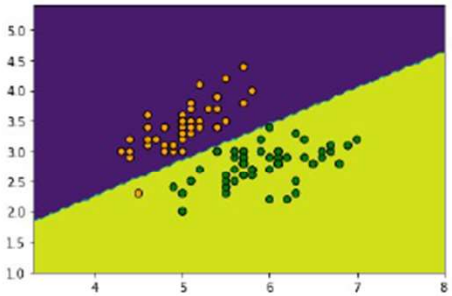
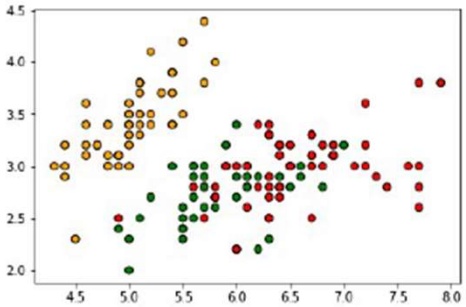
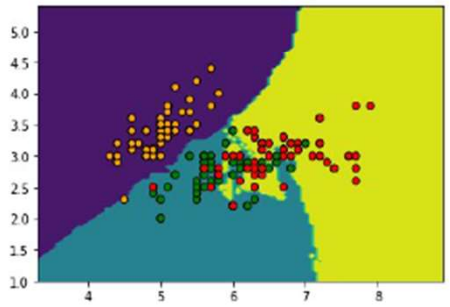
Para el **aprendizaje supervisado** se dispone de un **dataset etiquetado**.

Cuando el **atributo target es categórico**, se trata de un problema de **clasificación**, que puede ser:

- Un problema de **clasificación binaria**, si el atributo target u objetivo es dicotómico, es decir, solo toma dos valores distintos.

- Un problema de **clasificación multiclase**, si el atributo target u objetivo no es dicotómico, es decir, toma más de dos valores distintos.

Si el atributo **target es numérico**, se trata de un problema de **regresión**.

Aprendizaje supervisado, clasificación (Etiquetas categóricas)	**Clasificación binaria** (Dos clases)	Datos etiquetados 	Modelo 
	Clasificación multiclase (más de dos clases)	Datos etiquetados 	Modelo 

El **objetivo** del aprendizaje supervisado consiste en **encontrar un modelo que permita predecir los valores del atributo target a partir de los valores de los atributos predictores** (o features).

Para ello, se buscan **valores óptimos para los parámetros** de un modelo, de manera que, en conjunto, **las predicciones** para todas las observaciones del dataset **sean lo más similares** que sea posible **a los valores esperados**, **o etiquetas**.

Es decir, de manera que el ajuste sea el máximo posible o, dicho con otras palabras, que **el error sea el mínimo posible**.

El ajuste del modelo, es decir, **la búsqueda de valores para los parámetros** del modelo que minimizan el error, **se realiza mediante un proceso de entrenamiento** con el dataset.

No hay que **confundir los hiperparámetros** con los **parámetros** de un modelo.

- Los valores para los **hiperparámetros se fijan antes** del proceso de entrenamiento.

- Los valores para los **parámetros se obtienen como resultado del proceso de entrenamiento**. Son los valores que minimizan el error.

Para un **problema de clasificación**, los valores de la predicción del modelo, y el esperado, son categóricos y, por tanto, **solo importa si son iguales o si son diferentes.**

El **error** se calcula **comparando las predicciones** calculadas por el modelo para los datos de entrada **con los correspondientes valores reales**.

Una posible media es el **accuracy rate o tasa de aciertos**.

Esta es la **proporción de casos en que el valor predicho por el modelo coincide con el valor real** o esperado de la etiqueta.

Pero **existen otras medidas**, que estudiaremos en esta unidad.

Ya **se ha explicado en detalle** un tipo de modelo de clasificación: el **k-NN**, k nearest neighbours o k vecinos más próximos.

En esta unidad **se verán otros tipos de modelos de clasificación** y su implementación en Scikit-Learn.

2

Clasificadores en Scikit-Learn

Todas las clases que implementan algoritmos de clasificación en Scikit-Learn tienen **características comunes:**

PARÁMETROS DEL CONSTRUCTOR	n_jobs	Número de procesos o hilos de ejecución concurrentes se pueden usar para el cálculo. None (Valor por defecto): 1 procesador. -1: Se usan todas las CPUs disponibles.
MÉTODOS	fit(X,y)	Entrenar el clasificador con X (atributos predictores) e y (etiquetas).
	predict(X)	Obtener la predicción del modelo para X
	score(X, y)	Obtener accuracy score o tasa de aciertos.
ATRIBUTOS	classes_	Etiquetas de clases
	n_features_in_	Número de atributos predictores encontrados y utilizados por el método fit.
	feature_names_in_	Nombres de atributos encontrados y utilizados por el método fit.
	n_sambles_fit_	Número de observaciones (samples) en dataset utilizado para entrenamiento con fit.

La **selección de atributos predictores** puede **variar según el clasificador** empleado.

Mientras **algunos clasificadores** funcionan exclusivamente con atributos **numéricos**, **otros** requieren atributos **categóricos**.

Asimismo, **dependiendo del clasificador**, puede ser necesario realizar ciertos **preprocesamientos específicos** sobre los atributos para optimizar su rendimiento.

Por ejemplo, que para **k-NN es conveniente normalizar los valores** de los atributos predictores, que deben ser todos numéricos, para que tomen valores **entre 0 y 1**.

El **entrenamiento**, se hace con el método **fit(),** al que se le pasan valores para dos parámetros **X** (atributos predictores) e **y** (etiquetas o valores para el atributo target).

Después se realiza la **validación** utilizando el **método predict()** para obtener la predicción del clasificador para los datos de un conjunto de validación.

Estas predicciones **se comparan con los valores correctos para las etiquetas**, para **obtener una métrica** de la calidad de los resultados.

Por último, una vez realizado el ajuste del modelo, existen atributos que proporcionan información relevante acerca del modelo o de los datos proporcionados para el entrenamiento.

Cada clasificador tendrá sus propios atributos adicionales, en los que se puede obtener información adicional específica del mismo.

Otras características comunes a gran parte de los clasificadores de Scikit-Learn son las siguientes:

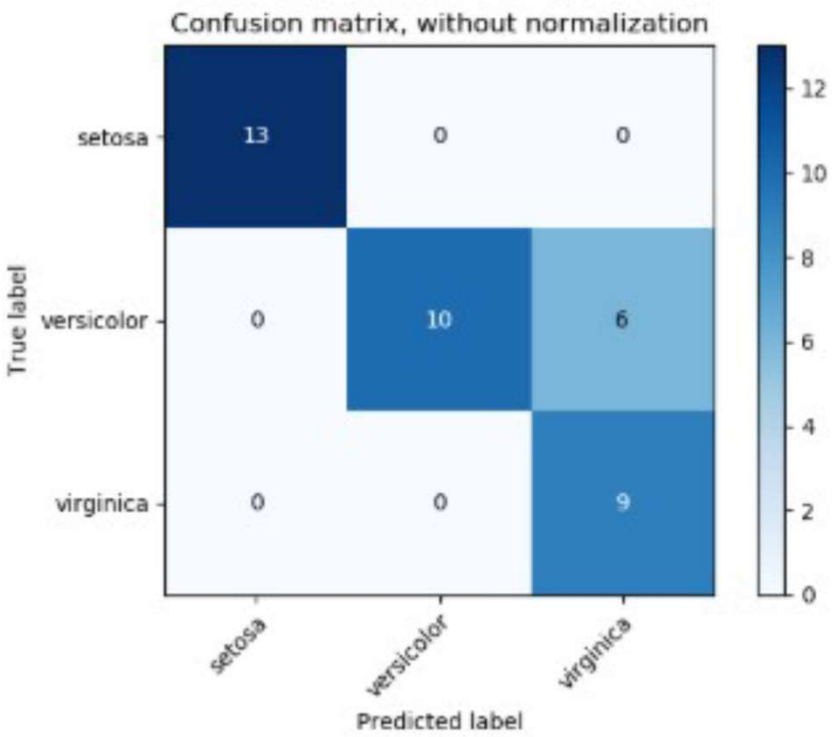
PARÁMETROS DEL CONSTRUCTOR	class_weight	Los pesos sirven para **ajustar la importancia de las observaciones de cada clase.** Las opciones comunes son: • **None**: Valor por defecto, todas las observaciones tienen el mismo peso. • **Diccionario**: Un diccionario que especifica el peso para cada clase. • **'balanced'**: Asigna un peso inversamente proporcional a la frecuencia de la clase en el conjunto de entrenamiento. Las clases menos frecuentes reciben más peso. Esto ayuda a los clasificadores a manejar datasets desequilibrados, siendo 'balanced' una opción recomendada en esos casos.
	sample_weight	**Pesos para observaciones particulares.** Se pueden usar conjuntamente pesos diferentes para clases diferentes (class_weight) y pesos diferentes para observaciones diferentes (sample_weight).
	random_state	**Semilla** para la generación de números pseudoaleatorios.
MÉTODOS	predict_proba(X)	**Obtener las probabilidades de que cada una de las observaciones** de X pertenezcan a cada una de las clases.

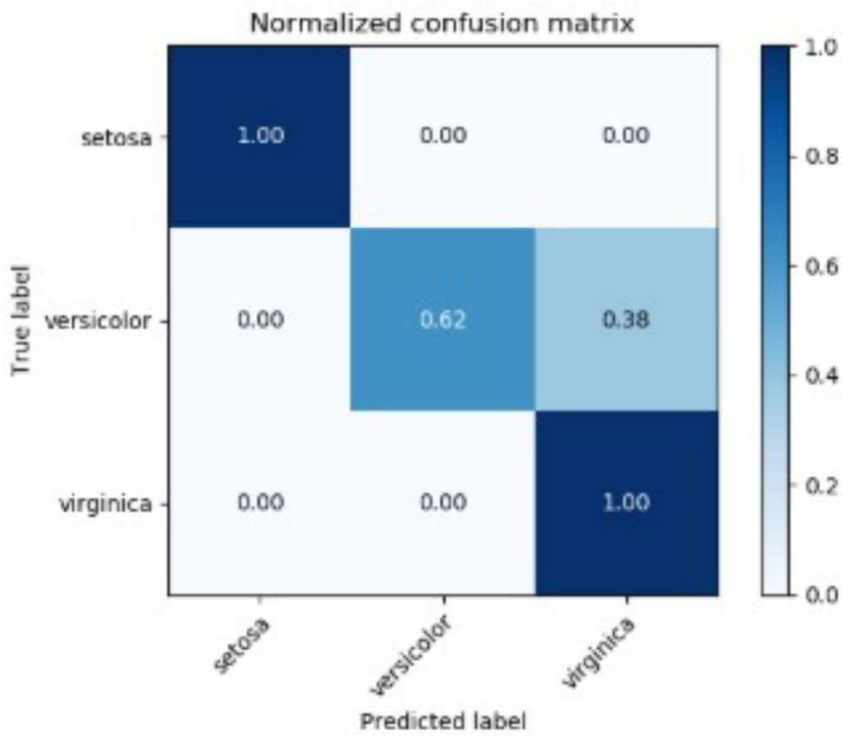
3

Matriz de Confusión

Una **matriz de confusión** es una matriz cuadrada que **contiene la frecuencia de cada posible combinación de valor predicho con valor real** para el atributo target.

Es decir, es una matriz de contingencia para los **valores reales y los valores predichos** del atributo target.





Las cantidades en la **diagonal principal** corresponden a **datos correctamente clasificados**, y **fuera de ella a datos incorrectamente clasificados**.

La matriz de confusión **se puede normalizar** por valores reales.

Para ello, en **cada fila**, correspondiente a un valor real de la etiqueta (true label), **se dividen todos los valores por la suma de los valores de la fil**a.

De esta forma, en **cada columna queda la proporción de datos** (de 0 a 1) con ese valor de la etiqueta que fueron clasificados con el valor indicado en la columna (predicted label).

Con un **clasificador perfecto**, en la **diagonal principal habría solo unos**, y fuera de ella solo ceros.

Los valores pueden multiplicarse por 100 para expresarlos en %.

El siguiente **código muestra la matriz de confusión** para un clasificador, a partir de los valores de las predicciones de un clasificador (en y_pred), y de los valores correctos (en y).

Las **etiquetas** las proporciona un clasificador previamente entrenado clf, en **clf.classes_**.

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
cm = confusion_matrix(y, y_pred, labels=clf.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=clf.classes_)
disp.plot()
```

Se puede normalizar la matriz de confusión por la clase verdadera ('**true**'), por clase predicha ('**pred**') o por el conjunto de los datos ('**all**').

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
cm = confusion_matrix(y, y_pred, labels=clf.classes_,
normalize='true')
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=clf.classes_)
disp.plot()
```

4

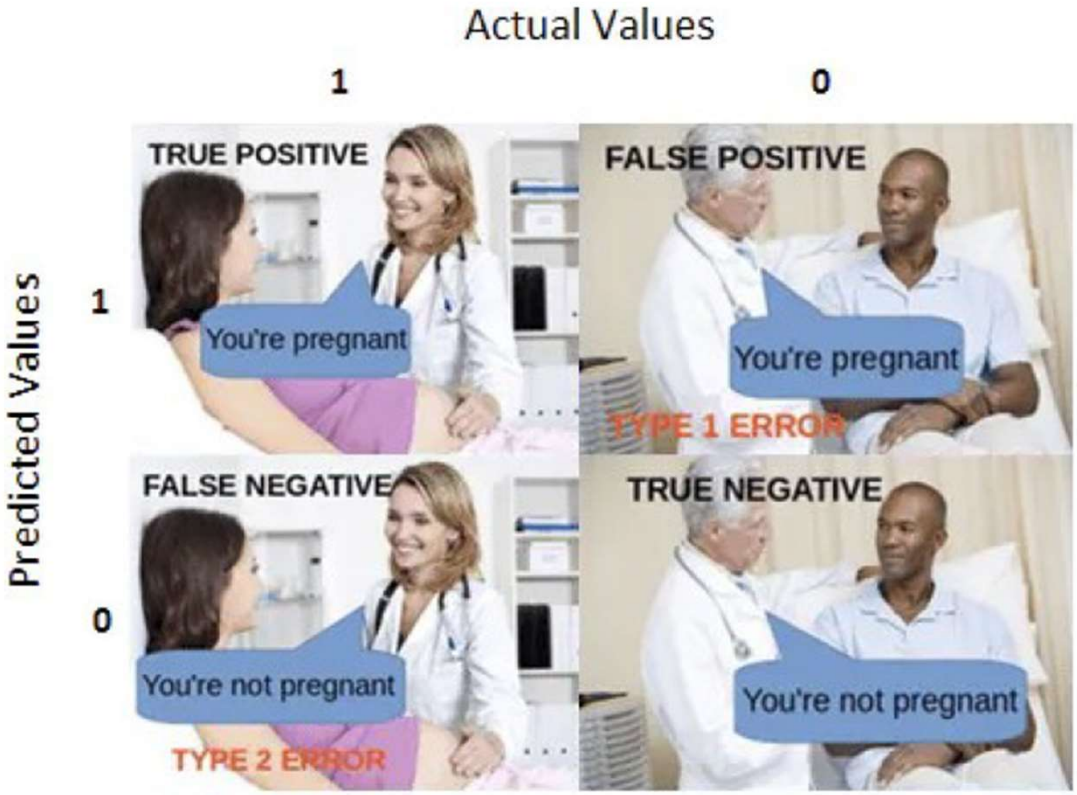
Métricas para Clasificadores Binarios

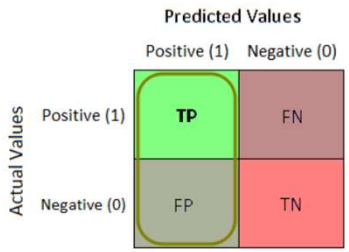
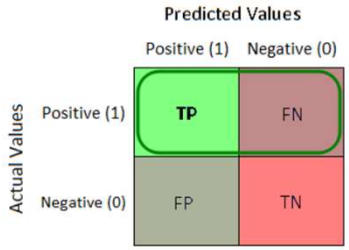
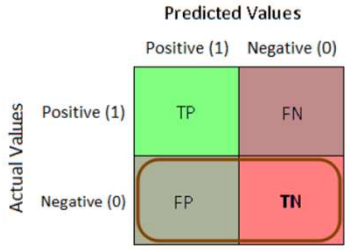
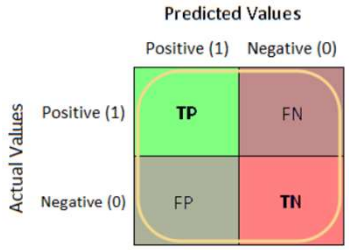
En el caso particular de clasificadores binarios que **clasifican los datos en dos categorías**, como positivos y negativos, **la matriz de confusión es una matriz de 2x2**, y se utiliza una terminología específica para cada una de sus cuatro posiciones.

En base a estos cuatro valores se definen diversas métricas.

		Predicted Values	
		Positive (1)	Negative (0)
Actual Values	Positive (1)	TP	FN
	Negative (0)	FP	TN

TP: *True positives* o verdaderos positivos.
TN: *True negatives* o verdaderos negativos.
FN: *False negatives* o falsos negativos.
FP: *False positives* o falsos positivos.
AP = TP+FN: *Actual positives* o positivos reales.
AN = TN+FP: *Actual negatives* o negativos reales.



Precision	Precisión	$\frac{TP}{TP+FP}$	Proporción de los clasificados como positivos que de hecho lo son.	
Sensitivity, recall, hit rate o true positive rate (TPR)	Sensibilidad	$\frac{TP}{TP+FN}$	Proporción de los positivos que han sido correctamente clasificados como tales.	
Specificity o true negative rate (TNR)	Especifidad	$\frac{TN}{TN+FP}$	Proporción de los negativos que han sido clasificados correctamente como tales.	
Accuracy	Exactitud	$\frac{TP+TN}{TP+FP+TN+FN}$	Proporción de observaciones correctamente clasificadas	

Precisión vs. sensibilidad o recall.

La **precisión** es la proporción de verdaderos positivos (TP) entre los clasificados como tales (TP+FP).

> Da una idea de la **probabilidad de que un dato clasificado como positivo lo sea realmente.**

La **sensibilidad** (recall) es la proporción de verdaderos positivos (TP) frente al total de positivos (TP+FN).

> Da una idea de la **probabilidad de que un dato positivo sea clasificado** como tal.

Si hay una **diferencia grande** entre el número de observaciones con valores del atributo target positivo (TP+FN) y negativo (TN+FP), se tiene un dataset **desequilibrado** (imbalanced dataset).

Por ahora se **supondrá que no es el caso**, es decir, que ambos están en torno al 50% de observaciones del dataset.

Entonces, un **clasificador puede tener tendencia a clasificar los casos dudosos como negativos o como positivos.**

En el caso en que los clasifique como negativos **aumentará FN** y **disminuirá FP**.

En el caso en que los clasifique como positivos **disminuirá FN** y **aumentará FP**.

En el **primer caso**, si el clasificador es riguroso a la hora de considerar como positivo un dato, su **precisión aumentará**, porque la gran mayoría de los que considera positivos lo serán (FP disminuirá).

Pero en contrapartida no estará considerando como tales a muchos positivos (FN aumentará), **y entonces su sensibilidad o recall disminuirá**.

Si el clasificador es menos riguroso para considerar como positivo un dato, está **favoreciendo la sensibilidad** o recall, porque FN disminuirá, pero en contrapartida FP aumentará, lo que irá en **detrimento de la precisión**.

Esto es lo que se conoce como **precision/recall tradeoff (contrapartida).**

En un **extremo**, se puede tener un **recall o sensitivity del 100%** clasificando todas las observaciones como positivas, porque entonces todas las observaciones positivas serán clasificadas como tales, y FN=TN=0, lo que por supuesto **irá muy en detrimento de la precisión.**

Y en el **extremo opuesto**, se puede tener una **especificidad del 100%** clasificando todas las observaciones como negativas, y una precisión del 100% o cercana con un clasificador muy exigente para clasificar observaciones como positivas, lo que irá en **detrimento del recall o sensitivity**.

Esto, en general, no tendrá sentido y **un clasificador de este tipo ni siquiera merecerá llamarse clasificador,** porque realmente no clasifica.

En general, **interesará un buen compromiso entre recall o sensitivity y precisión**.

Según la métrica que se tome como base para el proceso de entrenamiento, **se puede favorecer el recall o sensitivity, la precisión, o un compromiso** entre ambas.

Pero también, según la aplicación, **puede interesar más la precisión o la sensibilidad.**

Para un sistema de recomendación para una tienda electrónica, por ejemplo, se quiere asegurar que un producto recomendado a un usuario sea satisfactorio, aún a riesgo de no recomendar productos que podrían ser satisfactorios.

Lo que se quiere es ***evitar que el usuario ignore en el futuro las recomendaciones*** *debido a una mala experiencia.*

En un sistema de diagnóstico de enfermedades se quiere una alta sensibilidad, para ***evitar que ninguna enfermedad no sea diagnosticada****, aún a riesgo de diagnosticar como positivos a pacientes que realmente no tienen la enfermedad.*

Los **datasets desequilibrados** (imbalanced dataset) plantean muchas **dificultades**.

Para empezar, las **medidas no son significativas**, o no lo son todas.

En un **caso extremo** en que casi todas las **observaciones son positivas**, se puede obtener una **precisión cercana al 100%** simplemente clasificando como positivas todas las observaciones.

Por una parte, esto **no tiene sentido** y un sistema así no se puede siquiera considerar un clasificador.

Por otra parte, en ese caso, normalmente, **lo que interesa es detectar correctamente toda las observaciones negativas, pero con un recall o sensitividad aceptable**.

En el **caso opuesto**, cuando casi **todas las observaciones son negativas**, normalmente se **quiere clasificar correctamente todas las observaciones positivas**, pero con **valores aceptables para la precisión**.

La **dificultad intrínseca** que se tiene con estos datasets es que la **escasez de datos** de una de las clases **hace difícil el proceso de aprendizaje**.

En estos casos, se suelen **replicar las observaciones de la clase menos frecuente** para **limitar la descompensación**, por ejemplo hasta llegar a un **mínimo del 40%** para la clase menos representada.

Más adelante estudiaremos cómo hacer esto con el método **sample** de DataFrame.

Por otra parte, muchos clasificadores de Scikit-Learn tienen un **parámetro class_weight** para el constructor, que **permite asignar un peso diferente a las observaciones de cada clase.**

Por defecto, todas las observaciones tienen igual peso.

Con el valor '**balanced**', se asigna a cada observación un **peso inversamente proporcional a la frecuencia relativa** en el dataset de entrenamiento de la clase a la que pertenece.

Diversos **algoritmos utilizarán estos pesos de forma diversa**.

Pero con frecuencia, cuando se tiene un dataset desequilibrado, será recomendable especificar el valor 'balanced' para class_weight.

MEDIDA F

También llamada **f-score** o f1-score.

Combina precisión y sensibilidad o recall, es decir, es la media armónica entre ellas dos, es decir:

$$\frac{1}{\left(\frac{1}{precis}+\frac{1}{recall}\right)/2}$$

Operando, se llega a la siguiente expresión:

$$f_score = 2 \cdot \frac{precis \cdot recall}{precis + recall} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

En general, **un valor alto** de esta medida significa que tanto precisión como sensibilidad tienen valores altos, es decir, que se ha conseguido un **buen compromiso entre ambas.**